

IMY 220

Assignment 5

TailwindCSS and File Uploads

Due: 07 October at 13:00

The submission instructions are available on ClickUP. Any deviation from these instructions will result in a 10% deduction from your mark.

Instructions

This is an individual, take-home assignment.

In this assignment, you will extend a supplied PhotoShare application by adding **image upload functionality** and styling the React interface using **TailwindCSS**.

The supplied application already contains:

- a React client;
- an Express server;
- the basic PhotoShare post functionality;
- the required project structure.

You must extend the application so that users can upload an image when creating a new post.

Uploaded images must be processed by the Express server using **Multer** and stored inside the supplied uploads folder.

The uploaded images must then be accessible from the Express server and displayed as part of the relevant PhotoShare posts.

You must also use **Tailwind CSS v3 utility classes** to style the React application.

Complete the assignment using functional React components, not class components.

Ensure that your student number is included as a comment at the top of all files that you modify. Replace uXXXXXXX with your student number.

// Student Number: uXXXXXXX

Scenario

You are continuing development on the PhotoShare social media application.

PhotoShare currently allows users to create and view posts containing:

- a username;
- a caption.

The application must now support **image posts**.

When creating a new post, the user must provide:

- a username;
- a caption;
- an image.

The React application must send the post information and selected image to the Express server.

The Express server must use Multer to process and store the uploaded image.

The completed post, including its image, must then be displayed in the PhotoShare feed.

You must also redesign the supplied interface using TailwindCSS.

Supplied Project

Download and extract the supplied starter project.

Open the project folder in Visual Studio Code.

The project contains:

assignment-5/

```
|— client/
|  |— src/
|  |  |— components/
|  |  |  |— AddPostForm.jsx
```

```
| | | └─ PostList.jsx
| | | └─ Post.jsx
| | └─ App.jsx
| | └─ ...
| └─ package.json
|
└─ server/
    └─ uploads/
    └─ server.js
    └─ package.json
    └─ package-lock.json
```

The `client` folder contains the supplied React application.

The `server` folder contains the supplied Express API.

The `uploads` folder will be used to store uploaded images.

The supplied Express server uses an **in-memory array** to store post data. Posts added while testing will therefore be reset when the Express server is restarted. **You are not required to implement persistent storage for the post data.**

Uploaded image files are stored separately inside the `uploads` folder and will remain there when the Express server is restarted.

You may create additional files or components if required, but do not remove or substantially change the supplied functionality.

Section 1: Image Input

Update the supplied `AddPostForm` component so that a user can select an image when creating a post.

Add a file input:

```
<input type="file">
```

The file input must only be used to select the image for the post.

Store or retrieve the selected File so that it can be included when the form is submitted.

The form must require:

- a username;
- a caption;
- an image.

The application must not submit the post if the required values have not been provided.

Section 2: FormData

Update the post submission functionality so that the image can be sent to the Express API.

Create a new FormData object.

Add the following values to the FormData:

- username;
- caption;
- image.

For example, the image must be added using the field name:

image

Send the FormData to the supplied Express endpoint using `fetch()` and a POST request.

The FormData object must be used as the request body.

Do not convert the FormData to JSON.

After a post has been successfully created, clear the username, caption and image inputs so that the form is ready for a new post.

Section 3: Multer Configuration

Use the supplied **Multer** package in the Express server to process uploaded images.

Configure Multer using `diskStorage`.

The Multer configuration must specify:

- the destination for uploaded images;
- the filename used when storing an uploaded image.

Uploaded files must be stored inside the supplied:

uploads/

folder.

Create the required Multer upload middleware.

Section 4: Uploading an Image

Update the supplied POST route so that it can process the uploaded image.

Use Multer's:

`upload.single("image")`

middleware on the route.

The field name used by Multer must match the field name used when adding the image to the `FormData`.

The route must retrieve:

- the submitted username;
- the submitted caption;
- information about the uploaded image.

Create the new post using the supplied application structure.

The post must include sufficient image information for the React application to retrieve and display the uploaded image.

Return an appropriate response to the React application after the post has been created.

Section 5: Serving Uploaded Images

Configure Express so that uploaded images can be accessed by the React application.

Use:

```
express.static()
```

to serve the uploaded image files.

The uploaded images must be accessible through the Express server.

Do not embed uploaded image files directly inside the React application.

Section 6: Displaying Uploaded Images

Update the supplied Post component to display the uploaded image associated with each post.

Each post in the PhotoShare feed must display:

- the username;
- the uploaded image;
- the caption.

Use an `<img>` element to display the image.

Provide appropriate `alt` text for the image.

After successfully creating a new post, the new post and its uploaded image must appear in the PhotoShare feed.

Section 7: TailwindCSS

Configure the supplied React project to use **Tailwind CSS v3**.

You must use **Tailwind CSS v3** for this assignment, as this is the version used in the supplied lecture material.

Install the required Tailwind dependencies using:

```
npm install -D tailwindcss@3 postcss autoprefixer
```

Create the required Tailwind and PostCSS configuration files using:

```
npx tailwindcss init -p
```

Configure Tailwind to scan the supplied React files and add the required Tailwind directives to the project's CSS file.

Use **Tailwind utility classes** to style the PhotoShare interface.

Tailwind provides utility classes that can be combined to construct custom components rather than relying on predefined components.

Use TailwindCSS to style the following:

- the overall page layout;
- headings and text;
- the new-post form;
- labels and inputs;
- the file input;
- buttons;
- the PhotoShare feed;
- individual post containers;
- uploaded images.

Your styling must demonstrate appropriate use of:

- padding and margins;
- background and text colours;
- font sizes and weights;
- width and sizing;
- borders and/or border radius;
- layout and alignment utilities.

These correspond to the types of utility classes covered in the Tailwind lecture, including layouts, spacing, sizing, colours, typography, borders, radius and shadows.

The completed application must be readable and usable.

No default unstyled HTML elements should remain visible in the main PhotoShare interface.

You may use Tailwind's supplied colours, fonts, spacing and other utility classes.

You may customise the appearance of PhotoShare provided that all required functionality remains clearly usable.

Section 8: Interaction Styling

Use Tailwind modifiers to provide visual feedback for interactive elements.

At least one button must include a hover style.

At least one form control must include a focus style.

For example, your styling may change:

- colour;
- background colour;
- border;
- outline;
- shadow;

when the user interacts with the element.

Expected Application Behaviour

When the application first loads, the existing PhotoShare posts must be displayed.

The user must be able to enter a username and caption and select an image.

When the form is submitted:

User completes form



Image selected



FormData created



POST request



Multer processes image



Image saved to uploads/



Post created



New post displayed in React

The uploaded image must be accessible through the Express server and displayed with the relevant post.

The complete React interface must be styled using TailwindCSS.

Testing Your Application

Before submitting, test the complete application.

Confirm that:

- the React client starts successfully;
 - the Express server starts successfully;
 - existing posts are displayed;
 - a username can be entered;
 - a caption can be entered;
 - an image can be selected;
 - the form cannot be submitted without the required information;
 - the submission uses `FormData`;
 - the image is successfully received by Multer;
 - the image is saved inside the `uploads` folder;
 - the new post is created successfully;
 - the uploaded image is accessible through Express;
 - the uploaded image appears in the PhotoShare feed;
 - multiple posts can contain different uploaded images;
 - the application uses Tailwind CSS v3;
 - the application is readable and usable;
 - buttons and form controls provide appropriate interaction feedback.
-

Submission Files

Submit the complete Assignment 5 project.

Do not include any `node_modules` folders.

Your ZIP file should contain both the `client` and `server` folders:

assignment-5/

```
├── client/
|   ├── src/
|   ├── package.json
|   ├── package-lock.json
|   └── ...
|
└── server/
    ├── uploads/
    ├── server.js
    ├── package.json
    └── package-lock.json
```

Ensure that the uploaded image files required to demonstrate your application are included inside the uploads folder.

Submission Instructions

Delete the `node_modules` folders from both the `client` and `server` folders before creating your ZIP file.

ZIP the complete project folder using a tool such as 7-Zip or WinRAR.

Name the ZIP file:

`a5-{student_number}.zip`

Replace `{student_number}` with your student number.

Submit your ZIP file before the deadline to avoid a late submission.